RFC-0003 — Component Model

Status: Draft

Category: Core Architecture

Version: 0.1

---

1. Purpose

The Component Model defines the canonical representation of every electronic design within the Components Platform.

Everything else is derived from the Component Model.

---

2. The Model-First Principle

The Component Model is the source of truth.

The following are projections of the model:

- Component Language
- Board
- Voice Commands
- AI Interaction
- JSON Operations
- Python SDK
- Verilog
- SystemVerilog
- Documentation

None of these own the design.

---

3. Architecture

```
          Component Model

     /       |        |
```

```
Component  Board  Operations  API

Language  Language

     \      |       |      /

        Components Platform
```

---

4. The Component Model is NOT

The Component Model is not:

- a text file
- JSON
- YAML
- Verilog
- SystemVerilog
- Python
- a visual diagram

These are merely representations.

---

5. Model Responsibilities

The model owns:

- components
- instances
- ports
- buses
- interfaces
- hierarchy
- connectivity
- packages
- dependencies
- electrical semantics

The model does not own:

- coordinates
- colors
- icons

- themes
- windows
- zoom
- routing
- rendering

Those belong to Boards.

---

6. Component

Every circuit is represented as a graph of Components.

Components contain:

- identity
- type
- ports
- metadata
- package binding

---

7. Port

A Port has:

Logical identity

Physical identity

Direction

Electrical behavior

Aliases

Package mapping

---

8. Connection

A Connection expresses

relationship

not graphics.

Connections never contain:

line bends

Bezier curves

orthogonal routing

visual labels

---

9. Bus

A Bus is a first-class object.

A Bus is never represented as

eight unrelated wires.

---

10. Interface

Interfaces group related Ports.

Examples:

Address Bus

Data Bus

Memory Bus

UART

SPI

GPIO

---

11. Hierarchy

Every Component may contain other Components.

Hierarchy is unlimited.

---

12. Identity

Every object has a stable identity.

Identity never depends upon

screen position

file order

drawing order

renderer

---

13. Physical Mapping

Logical ports remain independent of packages.

Packages bind

logical ports

to

physical pins.

Changing package

does not change

logical meaning.

---

14. Component Language

Component Language

is

one textual representation

of the model.

---

15. Board

Board

is

one visual representation

of the model.

---

16. JSON Operations

Operations

modify

the model.

Operations never modify

rendering directly.

---

17. Python SDK

Python manipulates

the model

through Operations.

Python never edits

internal structures directly.

---

18. AI

AI never edits files.

AI edits

the Component Model.

The Platform

chooses

how to serialize those edits.

---

19. Versioning

Versions belong to the model.

Representations may evolve independently.

---

20. Export

Every exporter

reads

the Component Model.

Examples:

SystemVerilog

Verilog

SVG

PDF

Documentation

Future formats

---

21. Import

Importers

produce

Component Models.

Import never writes directly into

Platform internals.

---

22. Replaceability

Every representation is replaceable.

The model is not.

---

23. Principle

> Everything is a projection of the Component Model.